

PGR 210 - Natural Language Processing Part

Kristiania University College

By Huamin Ren

Huamin.ren@kristiania.no



Outline

- **1. *Singular value decomposition (SVD)***
- 2. Latent Dirichlet allocation (LDA)
- 3. Word2Vec

1. SVD

- The algebra behind LSA called singular value decomposition.
- LSA uses SVD to find the combinations of words that are responsible, together, for the biggest variation in the data.
 - ✓ Line up the axes (dimensions) in your new vectors with the greatest “spread” or variance in the word frequencies
- New basis vectors are generated; each dimension (axes) becomes a combination of word frequencies rather than a single word frequency.

SVD is an algorithm for decomposing any matrix into three “factors,” three matrices that can be multiplied together to recreate the original matrix.

1. SVD

$$W_{m \times n} \Rightarrow U_{m \times p} S_{p \times p} V_{p \times n}^T$$

m: number of terms in vocabulary

n: number of documents

p: number of topics (same as the number of words)

Truncating the topics

- Hint:

```
from sklearn.decomposition import TruncatedSVD
```

```
svd = TruncatedSVD(n_components=16, n_iter=100)  
svd_topic_vectors = svd.fit_transform(tfidf_docs)
```

Implement LSA using the same dataset (sms-spam.csv)

Outline

- 1. Singular value decomposition (SVD)
- **2. *Latent Dirichlet allocation (LDiA)***
- 3. Word2Vec

2. Latent Dirichlet allocation (LDiA)

- The Latent Dirichlet Allocation technique is a generative probabilistic model: each document is assumed to have a combination of topics - the latent topics contain a Dirichlet prior over them.
- Less recommended, unless on text summarization task.

Outline

- 1. Singular value decomposition (SVD)
- 2. Latent Dirichlet allocation (LDiA)
- **3. *Word2Vec***

6. Word2Vec

- Word2vec was first presented publicly in 2013 at the ACL conference. Word2vec embeddings were four times more accurate (45%) compared to equivalent LSA models (11%) at answering analogy question
- Word2vec could transform the natural language vectors of token occurrence counts and frequencies into a much lower-dimensional Word2vec vectors.

In this lower-dimensional space, you can do your math and then convert back to a natural language space.

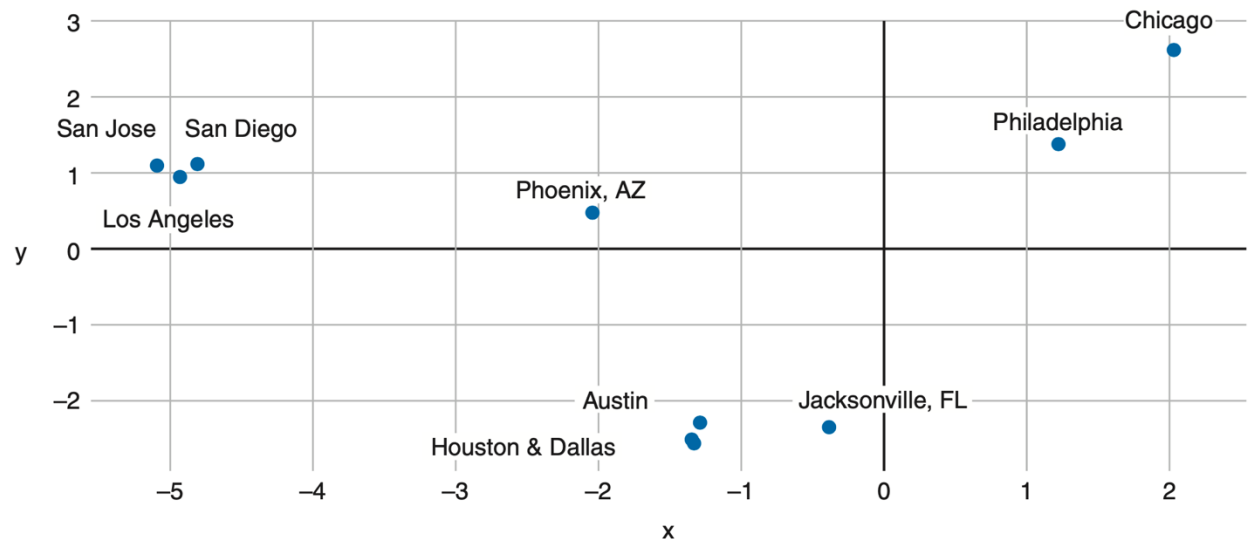
3. Word2Vec

$$\vec{x}_{coffee} - \vec{x}_{coffees} \approx \vec{x}_{cup} - \vec{x}_{cups} \approx \vec{x}_{cookie} - \vec{x}_{cookies}$$

Distance between the singular and plural versions of a word

3. Word2Vec – Reasons to use Word2Vec

- Useful for reasoning, analogy problems, pattern matching, modelling and visualization
- Can further do:
 - Visualize word vectors on a 2D semantic map
 - Great for chatbot and search engine



3. Word2Vec –

How to compute Word2Vec representations?

- Using `gensim.word2vec` module
 - Download pre-trained Word2vec models on Google News documents.
 - Limit the loaded words, if too memory intensive.
 - `gensim.KeyedVectors.most_similar()` method provides an efficient way to find the nearest neighbors for any given word vector.

A word vector model with a limited vocabulary will lead to a lower performance of your NLP pipeline if your documents contain words that you haven't loaded word vectors for.

3. Word2Vec – How to use pre-trained model?

- Using `gensim.word2vec` module
 - Download pre-trained Word2vec models on Google News documents.
 - Limit the loaded words, if too memory intensive.
 - `gensim.KeyedVectors.most_similar()` method provides an efficient way to find the nearest neighbors for any given word vector.

A word vector model with a limited vocabulary will lead to a lower performance of your NLP pipeline if your documents contain words that you haven't loaded word vectors for.

3. Word2Vec –

How to generate your own word vector representations?

- How to create your own domain-specific word vector models?

Keep in mind, you need *a lot of documents* to do this

3. Word2Vec – Look deeper into Word2Vec

- This model is a predictive deep learning based model to compute and generate high quality, distributed, and continuous dense vector representations of words that capture *contextual* and *semantic* similarity.
- Essentially unsupervised models: take in massive textual corpora, create a vocabulary of possible words, and generate dense word embeddings for each word in the vector space representing that vocabulary.
- The dimensionality of this dense vector space is much lower than the high-dimensional sparse vector space built using traditional Bag of Words models.

<https://arxiv.org/pdf/1301.3781.pdf>